

講義 28：客製 Table Maintenance 的權限防護與並行控制（授課順序：接在講義 27 之後）

對應練習：ex28 | 答案物件：表 ZTR28_CDISC + 權限物件 ZTR28_CARR + Lock Object EZTR28_CARR + T-code ZTR28_MAINT / ZTR28_SM30 + 程式 ZR_TR28_PARAM_MAINT / ZR_TR28_PRICE_CALC (ZR_TR28_PARAM_LIST 已棄用，見第 7 節說明)

本講重點

- 為什麼 SM30 原生的權限檢查不夠用——業務維度（如「只能改自己負責的航空公司」）要自己包一層
- 自訂權限物件（Authorization Object）：ACTVT + 業務欄位（本例 CARRID）
- Lock Object 的鎖定範圍可以比表格主鍵更粗——鎖「一家航空公司」而不是「一筆資料」
- VIEW_MAINTENANCE_CALL：從程式呼叫標準 Table Maintenance 的正規做法
- T-code 應該指到「包裝程式」，不是直接指到裸的維護畫面
- Report 選取畫面上的按鈕怎麼呼叫另一個 Transaction：SELECTION-SCREEN FUNCTION KEY + SE93 Parameter Transaction
- PFCG 角色維護：怎麼把自訂權限物件實際指派給一個使用者，讓 AUTHORITY-CHECK 真的能通過
- 主檔維護的參數不是憑空存在：另外寫一支計算報表（ZR_TR28_PRICE_CALC）JOIN 標準示範表 SPFLI/SFLIGHT，實際套用這裡維護的折扣算出最終票價，體會「維護畫面改的值，下游計算會馬上反映」

本講六項 GUI-only 步驟（SU21、PFCG 角色維護、SE11 Lock Object、SM30 Table Maintenance Generator、SE93 T-code ZTR28_MAINT、SE93 Parameter Transaction ZTR28_SM30）+ 兩支程式的 Selection Texts（第 10 節），全部都沒有 ADT 可以自動化，逐一附上完整操作路徑，照著做即可，不需要額外查資料。

1. 為什麼要包一層 Authorization Wrapper

講義 21 學過：SM30 產生的維護畫面背後靠 Table Maintenance Generator，存取權限預設是靠 S_TABU_DIS（Table Authorization Group）/ S_TABU_NAM（依表名）這兩個通用權限物件控管——它們回答的問題是「這個使用者能不能維護這張表」，是表格層級的粗粒度控管。

實務上常常需要更細的控管：「王小明只能改長榮航空(BR)的折扣，李小美只能改華航(CI)的折扣」——這種業務維度的權限，SM30 原生機制做不到（除非另外設定進階的 Organizational Criteria，那是更複雜的機制，本課不涉及）。

解法：不要把 T-code 直接指給 SM30，而是自己寫一支「包裝程式（Wrapper）」，T-code 指給這支程式。使用者要維護資料，一定先經過這支程式，程式裡先做：

1. 權限檢查（這個使用者對「這家航空公司」有沒有維護權限）
2. 上鎖（避免同一家航空公司同時有兩個人在維護，造成第 27 講學過的遺失更新）

兩關都過了，才呼叫標準機制把使用者帶進真正的維護畫面。這是企業導入客製主檔維護時的標準模式，值得完整走一次。

2. SM30 Table Maintenance Generator (跟講義 21 相同手法，欄位換成本題的表)

1. 先在 SE80 建 Function Group **ZFG_TR28B** (Table Maintenance Generator 只能放進已存在的 Function Group)
2. SE11 → 表 **ZTR28_CDISC** → Utilities → **Table Maintenance Generator**
3. Authorization Group **&NC&** (練習用，不檢核) ； Function Group 填 **ZFG_TR28B** ； Maintenance type 選 **One Step**
4. 產生後可以先直接用 **SM30** 手動測試維護畫面本身 (輸入 View 名稱 **ZTR28_CDISC** → Maintain)，確認能新增/修改一筆資料——這一步只是確認 View 本身能動，還沒有套用後面的權限/鎖定 Wrapper，兩者是獨立的

3. 自訂權限物件 (Authorization Object) : SU21

1. 交易碼輸入 **SU21** → Enter
2. 左側樹狀選單找一個 Object Class ( **BC** 這個代碼本身不存在，是分類的字首不是完整代碼，實測 F4 選單裡查得到的是 **BC_A** (Basis: Administration)、**BC_C** (Basis - Development Environment)、**BC_Z** (Basis - Central Functions) 等更細的子分類——本例選 **BC_A**；實務上依表格所屬模組選對應 Class，例如 PP 模組相關的表可以選 **PP**) → 對該 Class 按滑鼠右鍵 → **Create** (或工具列的「Create」按鈕)
3. **Object** 欄位輸入 **ZTR28_CARR** (權限物件名稱上限只有 10 碼，剛好符合——這點跟 Lock Object / 表格 / Data Element 動輒 16~30 碼的限制不同，10 碼是權限物件特有的較嚴格限制)，**Text** 填說明 (如「TR28 航空公司折扣維護權限」) → Enter
4. **Authorization Fields** 頁籤，逐一加兩個欄位 (每個欄位按 **Insert Row** 或直接在空白列輸入)：
 - **ACTVT** (標準欄位，Activity，代表「做什麼」：**01**=Create、**02**=Change、**03**=Display.....SAP 標準的活動代碼清單)
 - **CARRID** (本例的業務欄位，代表「對誰」：哪家航空公司——欄位型別會自動帶出 Data Element **S_CARR_ID** 的意義，因為 **CARRID** 是 SAP 標準保留的欄位名稱)
5. 存檔 (跳出的 Transport 對話框選 **Local Object**，練習用途不用建正式傳輸單)
6. 工具列 **Activate** (或 Ctrl+F3) ——存檔時如果看到「Permissible activities not maintained for field ACTVT」這種黃色警告 (不是紅色錯誤) 可以先忽略，那是提醒「還沒設定這個物件允許哪些 ACTVT 值」，不影響本題後續使用；如果是紅色的「Object class ... does not exist」就要回頭檢查 Class 代碼有沒有打對

ACTVT 幾乎是所有自訂權限物件的標配：光看「使用者對某張表有沒有權限」不夠，還要分「只能看」還是「可以改」。標準活動代碼 **01/02/03/06** (刪除) / **08** (顯示變更文件)...可以查 **SU21** 或 **SE11** 顯示 Data Element **ACTVT** 的固定值清單。

 本例的權限物件 **ZTR28_CARR** (無 **S**) 跟 Lock Object **EZTR28_CARR** (有 **S**，且 **E** 開頭) 拼法相近但不同，不要看錯——兩者只差開頭的 **E**，操作時務必看清楚。

程式裡用 **AUTHORITY-CHECK** 呼叫這個物件：

```
AUTHORITY-CHECK OBJECT 'ZTR28_CARR'
  ID 'ACTVT'  FIELD lv_actvt
  ID 'CARRID' FIELD p_carrid.

IF sy-subrc <> 0.
```

```
" 沒有權限—sy-subrc 常見值 ( 官方定義 ) : 4=使用者有這個物件的授權，但值對不上 ( 或欄位規格錯誤 ) ;
" 12=使用者的 User Master Record 裡完全沒有這個物件的任何授權 ( 不是「物件不存在」，是「這個人一筆都沒有」，
" 通常代表 PFCG 角色還沒指派給這個人，或角色沒 Generate/沒 User Comparison )
ENDIF.
```

權限物件只是定義了「有哪些欄位可以管控」，真正「誰對什麼值有權限」要靠角色維護 (PFCG) ——建好權限物件之後，還沒有任何人真的擁有這個權限，AUTHORITY-CHECK 一律會失敗，直到走完下面第 3.1 節的 PFCG 流程為止。

3.1 PFCG 角色 (Role) 維護：把權限物件真正指派給使用者

這一步是整套機制真正生效的關鍵，很多課程或文件會跳過，但沒有這一步，前面 SU21 建的物件形同虛設。完整走一次：

1. 交易碼輸入 **PFCG** → Enter
2. **Role** 欄位輸入角色名稱 (如 **ZTR28_MAINT_ROLE**，Z 開頭) → **Single Role** 按鈕 (不是 Composite Role)
3. **Description** 頁籤：Description 欄位填角色說明 (如「TR28 航空公司折扣維護角色」) → 存檔
4. **Menu** 頁籤 (讓角色使用者可以直接從選單找到這支交易，非必要但建議做) :
 - 工具列 **Transaction** 按鈕 (或右鍵 → Insert Transaction)
 - 輸入 T-code **ZTR28_MAINT** → Enter，選單樹會出現這一項
5. **Authorizations** 頁籤 → 按鉛筆圖示 **Change Authorization Data** :
 - 系統會自動帶入 T-code 本身需要的標準權限物件 (如 **S_TCODE**，因為 Menu 頁籤加了 **ZTR28_MAINT**)，但不會自動帶入我們自訂的 **ZTR28_CARR**——自訂物件沒有跟 T-code 自動關聯，要手動加
 - 工具列 **Manually** (手動) 按鈕 → 輸入 **ZTR28_CARR** → Enter，這個物件的節點會出現在權限樹狀清單裡
 - 展開 **ZTR28_CARR** 節點，把 **ACTVT** 欄位的值改成 **02** (Change；如果也想順便給顯示權限可以再加一行 **03**)
 - **CARRID** 欄位填 **LH** (想授權的航空公司代碼；教學上建議先只給 **LH**，具體體會「只能改自己負責的航空公司」的效果——真的要開放全部航空公司可以填 *)
 - 檢查 **S_TCODE** 節點底下的 **TCD** 欄位確實已經有 **ZTR28_MAINT**
6. 工具列齒輪圖示 **Generate** (產生 Profile) → 跳出的 Profile 名稱視窗直接 Enter 接受系統預設值
7. 回到角色主畫面 (可能要按左上角綠色返回箭頭) → **User** 頁籤 → **User ID** 欄位輸入自己的使用者代號 → Enter
8. 工具列 **User Comparison** (使用者比對，通常是兩個人形疊在一起的圖示，或選單 Utilities → User Comparison) → 跳出畫面按 **Complete Comparison**
9. 存檔

驗證方式：登出重新登入 (或另開一個新 Session/Mode)，執行 T-code **ZTR28_MAINT** (**p_carrid=LH**)，應該看到「權限檢查通過」而不是先前的「權限不足」。如果還是失敗，交易碼 **SU53** (顯示上一次權限失敗的畫面) 是排查權限問題最直接的工具，可以直接看到是哪個物件、哪個欄位值沒過。

4. Lock Object 的鎖定範圍可以比主鍵更粗

講義 27 的 **EZTR21_STUD** 鎖 **MANDT+ID** (表格完整主鍵)，意思是「鎖一筆學生資料」。本題的表 **ZTR28_CDISC** 主鍵是 **MANDT+CARRID** (兩個欄位)，Lock Object **EZTR28_CARR** 的 Lock Parameters 剛好就是這兩個欄位——因為本題的業務需求本來就是「同一家航空公司不能有兩人同時維護折扣」，鎖定範圍跟主鍵一致，不需要像講義 27 或 ex28 舊版那樣刻意鎖得比主鍵粗。

鎖定範圍要對應到「使用者實際上在爭搶的資源邊界」，不是機械式地照抄表格主鍵，也不是為了跟前一題不一樣而刻意設計得更粗——本題剛好主鍵範圍跟業務邊界一致，就直接鎖整個主鍵；如果之後表格多了第三個 Key 欄位 (例如「幣別」)，要重新思考鎖定範圍該不該涵蓋它。

SE11 建立步驟 (跟講義 27 相同流程，範圍不同)：

1. 交易碼輸入 **SE11** → 左側選 **Lock Object** → 輸入 **EZTR28_CARR** (系統強制規定要 **E** 開頭，不是單純的命名慣例——打別的字首存檔會直接跳出警告「Start the lock object names with the prefix 'E'」。 這裡容易跟第 3 節建的權限物件 **ZTR28_CARR** 搞混，兩者外觀相似但拼法不同 (**ZTR28_CARR** 無 **S** / **EZTR28_CARR** 有 **S**)，輸入時務必看清楚) → **Create**
2. **Tables** 頁籤：Primary Table 填 **ZTR28_CDISC** → Enter (系統會自動帶出這張表的完整欄位清單)
3. **Lock Parameters** 頁籤：勾選 **MANDT**、**CARRID** 兩個欄位的 **Lock parameter** 核取方塊
4. **Lock Mode** 欄位選 **E** (Exclusive/Write Lock)
5. 存檔 (Local Object)、工具列 **Activate**
6. 查詢系統自動產生的 **FM**：該畫面 → Utilities → Generated Objects，應該看到 **ENQUEUE_EZTR28_CARR** / **DEQUEUE_EZTR28_CARR**；也可以 SE37 直接 Display 這兩個 FM，確認 Import 參數是 **MANDT** / **CARRID**

5. **VIEW_MAINTENANCE_CALL**：從程式呼叫標準 Table Maintenance

土法煉鋼的做法是在程式裡 **CALL TRANSACTION 'SM30'**，但這樣沒辦法乾淨地指定要維護哪個 View、也繞不過 SM30 自己的初始畫面。SAP 提供了正規的函式模組：

```
CALL FUNCTION 'VIEW_MAINTENANCE_CALL'
  EXPORTING
    action      = 'U'          " U=維護 ( Update )、S=只顯示 ( Show )、T=帶Transport單維護
    view_name   = 'ZTR28_CDISC'
  EXCEPTIONS
    client_reference      = 1
    foreign_lock          = 2
    invalid_action        = 3
    no_clientindependent_auth = 4
    system_failure        = 5
    OTHERS                = 6.
```

這支 FM 內部會再做一次它自己的權限檢查 (**S_TABU_DIS**/**S_TABU_NAM**) 跟再上一次它自己的鎖 (**VIEW_ENQUEUE**，鎖的是整個 View，不是我們自訂的 Lock Object)——這是完全獨立、疊加在我們自訂 Wrapper 之上的第二層保護，兩層互不衝突：我們的 Wrapper 負責業務維度 (航空公司)，**VIEW_MAINTENANCE_CALL** 自己負責表格層級的通用維護權限與鎖定。

action = 'S' (只顯示) 搭配 **ACTVT = '03'** 的權限檢查，可以做出一個「只能看、不能改」的模式，這正是本題 Wrapper 程式 **p_disp** 核取方塊要做的事。

5.1 用 `dba_sellist` 篩選：只顯示使用者被授權的那家航空公司

只做完 `AUTHORITY-CHECK` 還不夠——它只回答「這個人能不能對航空公司 `LH` 做維護」，不會自動限制他進了 `VIEW_MAINTENANCE_CALL` 畫面之後只看得見 `LH` 的資料。如果不額外處理，一個只被授權 `CARRID=LH` 的使用者，一樣可以在維護畫面裡看到、甚至改到其他航空公司的列——這是很容易漏掉的一步。

`VIEW_MAINTENANCE_CALL` 有一個 `TABLES dba_sellist` 參數（型別 `VIMSELLIST`），可以帶入篩選條件，效果類似幫這次維護動作先套一個 `WHERE` 子句：

```
DATA: lt_sellist TYPE STANDARD TABLE OF vimsellist,
      ls_sellist TYPE vimsellist.

CLEAR ls_sellist.
ls_sellist-viewfield = 'CARRID'.    " 要篩選的欄位名稱 ( 大寫 )
ls_sellist-operator  = 'EQ'.        " 比較運算子：EQ/NE/GT/GE/LT/LE/LK ( Like )
ls_sellist-value     = p_carriid.   " 要比對的值
ls_sellist-tabix     = 1.           " 條件的序號 ( 多筆條件用 and_or 欄位串接 AND/OR )
APPEND ls_sellist TO lt_sellist.

CALL FUNCTION 'VIEW_MAINTENANCE_CALL'
  EXPORTING
    action      = 'U'
    view_name   = 'ZTR28_CDISC'
  TABLES
    dba_sellist = lt_sellist
  EXCEPTIONS
    ...
```

帶了這個篩選之後，維護畫面一開啟就只會列出 `CARRID = p_carriid` 這家航空公司的資料列，使用者不會不小心看到或改到別的航空公司——這一步刻意跟下一節 `ZR_TR28_PARAM_LIST` 那顆「透過 **Parameter Transaction** 呼叫 **SM30**」的按鈕做對照：那顆按鈕完全沒有篩選，會看到整張表所有航空公司的資料，兩者的差異正好示範了「隨便呼叫 **SM30**」跟「透過 **Wrapper** 正規呼叫」在資料曝露範圍上的落差。

⚠ `VIEW_MAINTENANCE_CALL` 會真的開出一個完整畫面，等使用者操作完才會返回——這代表它沒辦法用 ADT 的無頭 `programrun` API 驗證（`headless` 呼叫會卡在等畫面回應，最終連線逾時斷線 `RFC_CLOSED`），只能請使用者在真實 SAP GUI 測試，跟前面題目遇過的 `Smartform` 是同一類限制。還有一個連帶影響：如果 `Wrapper` 程式在呼叫這支 FM「之前」已經成功 `ENQUEUE_EZTR28_CARR`，`programrun` 卡在這裡逾時斷線時，程式不會執行到後面的 `DEQUEUE_EZTR28_CARR`（因為程式根本沒跑完），理論上鎖會殘留——**SM12** 跟 **Lock Object** 的關係、以及這種殘留鎖該怎麼排查與清除，見講義 **27 第 7 節**，那一節已經用這個真實情境當例子寫進去了。

6. T-code 要指給 `Wrapper`，不是指給裸的 **SM30**

- 交易碼輸入 **SE93** → **Transaction Code** 欄位輸入 `ZTR28_MAINT` → **Create**
- Short Text 填說明（如「**TR28** 航空公司折扣維護」）
- 型態選 **Program and selection screen (Report transaction)**（單選按鈕）→ Enter

4. **Program** 欄位填 `ZR_TR28_PARAM_MAINT` (我們的 Wrapper 程式，不是 SM30、也不是直接填 View 名稱)，**Selection Screen** 欄位填 `1000` (一般報表選取畫面的慣例編號，見 Enhancement 課程 en02 的說明)
5. 存檔 (Local Object)

這是整個模式能不能生效的關鍵一步：如果貪方便，直接開一個 T-code 指給 SM30、或是把 `S_TABU_DIS/S_TABU_NAM` 開放給所有人，使用者就能繞過我們寫的 Wrapper 直接進 SM30 維護，前面做的權限檢查與 Lock Object 就完全形同虛設。實務上要搭配：一般使用者的角色不給 **SM30 / 裸 View** 維護的權限，只給這支 **Wrapper** 的 **T-code**，才能確保大家只能走這條有檢查的路。

7. 選取畫面上的按鈕：SELECTION-SCREEN FUNCTION KEY

⚠ **2026-07-31 更新**：這個按鈕原本示範在一支獨立的「清單程式」`ZR_TR28_PARAM_LIST` 上，但那支程式的唯一用途就是展示這顆按鈕，跟主要報表 `ZR_TR28_PRICE_CALC` 的按鈕功能完全重複，教學上多餘，已拿掉——這顆按鈕現在直接掛在 `ZR_TR28_PRICE_CALC` (主要的折扣計算報表) 的選取畫面上。

`ZR_TR28_PRICE_CALC` 是「主要報表」——它的選取畫面 (輸入 `p_carriid / p_connid` 的那個畫面) 上直接加一顆按鈕，讓使用者不用先跑出報表結果、也不用另外去記 T-code，就能捷徑跳去維護折扣主檔。這跟 ALV 工具列按鈕 (講義 9 進階篇，`IT_EVENTS`)、或更早版本一度用過的清單畫面 `SET PF-STATUS/AT USER-COMMAND` (那是掛在執行完畢後的輸出清單上) 都不一樣——這裡要按的是選取畫面本身的按鈕，語法是 **SELECTION-SCREEN FUNCTION KEY**：

1. 宣告 **TABLES: sscrfields.**——這是系統結構，專門用來承接選取畫面上的功能鍵被按下時的資訊 (哪一個按鈕、代碼是什麼)
2. 宣告功能鍵：**SELECTION-SCREEN FUNCTION KEY 1.** (位置很重要，這一行要放在選取畫面相關宣告區塊，即 `PARAMETERS/SELECT-OPTIONS` 附近，不能放在事件區塊裡) ——最多可以宣告到 **FUNCTION KEY 4** (四顆自訂按鈕)，系統固定把它們的 Function Code 命名為 `'FC01' / 'FC02' / 'FC03' / 'FC04'`，這四個代碼是系統保留的，不能自己改
3. 在 **INITIALIZATION** 事件裡設定按鈕文字：`sscrfields-functxt_01 = '維護主檔(SM30)'`。(`functxt_01` 對應 **FUNCTION KEY 1**；有幾個按鈕就對應填 `functxt_01 ~ functxt_04`)
4. 在 **AT SELECTION-SCREEN** 事件裡判斷按鈕被按下、處理動作：

```
TABLES: sscrfields.

PARAMETERS: p_carriid TYPE spfli-carriid DEFAULT 'LH' OBLIGATORY,
             p_connid TYPE spfli-connid DEFAULT '0400' OBLIGATORY.

SELECTION-SCREEN FUNCTION KEY 1.

INITIALIZATION.
    sscrfields-functxt_01 = '維護主檔(SM30)'.

AT SELECTION-SCREEN.
    CASE sscrfields-ucomm.
        WHEN 'FC01'.
            CALL TRANSACTION 'ZTR28_SM30'.
    ENDCASE.
```

ZTR28_SM30 不是硬寫在程式碼裡的 SET PARAMETER / CALL TRANSACTION 'SM30' AND SKIP FIRST SCREEN，而是 SE93 建立的正規 **Parameter Transaction**，把「要維護哪個 View、要不要跳過初始畫面」這些設定搬到 T-code 定義層級，呼叫端只要單純 CALL TRANSACTION。建立步驟：

- 1. SE93 → Transaction Code 輸入 ZTR28_SM30 → Create
- 2. Transaction Text 填說明 (如「M30 維護 ZTR28_CDISC (Parameter Transaction)」)
- 3. 型態選 **Transaction with parameters (parameter transaction)**
- 4. **Default values for** → **Transaction** 填 SM30
- 5. 勾選 **Skip initial screen**
- 6. ⚠️ 不是填上方「Values for SPA/GPA Parameters」表格——正確位置是畫面下方獨立的 **Default Values** (欄位標題「Name of screen field」/「Value」) 表格，新增：
 - VIEWNAME = ZTR28_CDISC
 - UPDATE = X
- 7. 存檔 (Local Object / \$TMP)

VIEWNAME 是 SM30 初始畫面「Table/View」欄位的實際技術欄位名稱，UPDATE = X 讓畫面直接進入維護 (更新) 模式而不是唯讀顯示——兩者都是直接指定 SM30 那張畫面上的欄位值，跟「Values for SPA/GPA Parameters」(先把值塞進 SPA/GPA 記憶體、由畫面自己讀取) 是兩種不同機制，SM30 這個情境要用前者才會生效。

講義 13 的傳票清單 ZRFI0004 也是這個寫法：選擇畫面的兩顆按鈕 CALL TRANSACTION 'ZFI0037' / 'ZFI0037Q'，就是用 Parameter Transaction 開啟會計主管表 ZFI0037 的 SM30 維護 / 查詢畫面。當時只要求看懂，到這裡才實際建立。它的查詢版還多做一步：ZFI0037Q 開的不是表本身，而是 Maint. Status 設成 Read only 的 Maintenance View。原因是 UPDATE 留空讓 SM30 以顯示模式開啟，有維護權限的使用者還是能按 Change / Display 切換去改資料；唯讀 View 根本沒有這顆切換按鈕。

⚠️ 這顆按鈕故意示範「沒有保護」的做法，跟 ZR_TR28_PARAM_MAINT 對照

這裡刻意不透過 ZR_TR28_PARAM_MAINT 那個 Wrapper，而是透過 ZTR28_SM30 這個 Parameter Transaction 直接跳進 SM30——這是為了跟 Wrapper 的正規做法做對比教學，兩者差在：

	ZR_TR28_PRICE_CALC 的按鈕	ZR_TR28_PARAM_MAINT (T-code ZTR28_MAINT)
呼叫方式	CALL TRANSACTION 'ZTR28_SM30' (Parameter Transaction 包了 SM30，但本身不含任何檢查)	走完整的 Wrapper 流程
業務權限檢查 (ZTR28_CARR)	✗ 沒有	✓ 有
Lock Object (EZTR28_CARR)	✗ 沒有	✓ 有
資料篩選 (只看單一航空公司)	✗ 沒有，看得到整張表所有航空公司	✓ 有 (見上一節 dba_sellist)
適合場景	純示範對照用；實務上若真的要開放，只能給完全信任、本來就有 SM30/S_TABU_DIS 權限的維護人員	一般使用者的正式入口

這正是本題最重要的教學重點：同樣是「從另一支程式跳進主檔維護」，寫法上的差別（有沒有包一層 Wrapper）決定了資安上是天壤之別。實務上兩個入口不會同時開放給一般使用者——會像講義第 6 節說的，只把 T-code **ZTR28_MAINT** 的權限給一般使用者，**ZR_TR28_PRICE_CALC** 這顆按鈕的存在本身就是提醒學員「這樣寫是不對的」，不是真的建議這樣上線；正因為這顆按鈕已經足夠示範對照，不需要再另外寫一支專門的清單程式重複同一件事。

7.1 按鈕加上圖示：SAP 圖示 (Icon) 的使用

實務程式的按鈕常常是「圖示 + 文字」，例如講義 13 的傳票清單 ZRFI0004：

```
DATA gs_funcxt TYPE smp_dyntxt.

INITIALIZATION.
  gs_funcxt-icon_id    = icon_tools.      " 圖示：工具
  gs_funcxt-icon_text = text-t03.        " 圖示旁邊的按鈕文字
  sscrfields-functxt_01 = gs_funcxt.
```

圖示是什麼：一個 4 字元的代碼

SAP 的每個圖示都有一個 4 個字元的內部代碼，格式是 @xx@。SAP GUI 在畫面上遇到這種代碼，就把它畫成對應的圖案。系統裡的圖示登記在表 **ICON**，本系統共 1,230 個。實際查到的幾個例子：

常數名稱	代碼	圖示
icon_green_light	@08@	綠燈
icon_yellow_light	@09@	黃燈
icon_red_light	@0A@	紅燈
icon_okay	@0V@	勾勾 (確定)
icon_cancel	@0W@	叉叉 (取消)
icon_change	@0Z@	鉛筆 (修改)
icon_display	@10@	眼鏡 (顯示)
icon_tools	@45@	工具

程式裡不要直接寫 '@45@'，要用常數 **icon_tools**：名稱看得懂，而且 SAP 保證常數跟代碼永遠對得上。這些常數定義在 Type Group (型別群組) **ICON** 裡。舊程式會先寫 **TYPE-POOLS: icon.** 才能用；ABAP 7.02 之後系統會自動載入，不寫也可以 (本系統實測，沒寫 **TYPE-POOLS** 也能正常啟用)。看到舊程式的 **TYPE-POOLS** 保留無妨，新程式可以省略。

怎麼找到想要的圖示

- 交易碼 **ICON**：列出所有圖示的圖案、名稱與代碼，可依名稱搜尋，最方便。
- 報表 **SHOWICON** (SE38 執行)：同樣列出全部圖示。
- 找到後，名稱轉成小寫就是常數名稱，例如 **ICON_EXECUTE_OBJECT** → **icon_execute_object**。

用法一：選擇畫面工具列按鈕 (**FUNCTION KEY**)

sscrfields-functxt_01~04 除了直接放文字 (本節前面的寫法) ，也可以放一個 **SMP_DYNTXT** 結構，一次給圖示、文字和滑鼠提示：

SMP_DYNTXT 欄位	用途
icon_id	圖示代碼，填 icon_xxx 常數
icon_text	按鈕上圖示旁邊的文字
quickinfo	滑鼠停在按鈕上時的提示文字

```
DATA gs_functxt TYPE smp_dyntxt.
```

```
INITIALIZATION.
```

```
gs_functxt-icon_id    = icon_tools.  
gs_functxt-icon_text  = '工具'.  
gs_functxt-quickinfo  = '開啟維護工具'.  
sscrfields-functxt_01 = gs_functxt.
```

```
CLEAR gs_functxt.                                " 第二顆按鈕前先清空，不然會帶到上一顆的提示  
gs_functxt-icon_id    = icon_display.  
gs_functxt-icon_text  = '查詢'.  
sscrfields-functxt_02 = gs_functxt.
```

ZRFI0004 設第二顆按鈕前先 **CLEAR functxt**，就是這個原因：同一個結構重複使用，沒清空的欄位會沿用上一顆的值。

用法二：選擇畫面上的按鈕 (**PUSHBUTTON**) 與 **ICON_CREATE**

選擇畫面本身也可以放按鈕。按鈕文字欄位要同時放圖示和文字時，用標準 FM **ICON_CREATE** 組出來：

```
SELECTION-SCREEN PUSHBUTTON /1(20) b_run USER-COMMAND zrun.    " b_run 是按鈕文字欄位
```

```
INITIALIZATION.
```

```
CALL FUNCTION 'ICON_CREATE'
```

```
EXPORTING
```

```
name           = icon_execute_object    " 圖示  
text           = '執行檢查'             " 圖示旁的文字  
info           = '依等級顯示結果'      " 滑鼠提示
```

```
IMPORTING
```

```
result         = b_run
```

```
EXCEPTIONS
```

```
icon_not_found      = 1  
outputfield_too_short = 2  
OTHERS              = 3.
```

```

IF sy-subrc <> 0.
    b_run = '執行檢查'.           " 組不出來就只放文字
ENDIF.

AT SELECTION-SCREEN.
    IF sscrfields-ucomm = 'ZRUN'.   " PUSHBUTTON 的代碼是自己取的 USER-COMMAND
        ...
    ENDIF.

```

ICON_CREATE 組出來的內容長這樣 (實測) : `@0V\Q全部資料已處理@處理完成` , 也就是「@ + 圖示代碼 + \Q + 提示文字 + @ + 顯示文字」。SAP GUI 看得懂這個格式 , 會畫成「圖示 + 文字 , 滑鼠停上去顯示提示」。所以目標欄位要夠長 , 放不下時 FM 會回 `outputfield_too_short` 。

用法三：清單輸出圖示 (**WRITE ... AS ICON**)

報表常用紅黃綠燈表示狀態 , **WRITE** 加 **AS ICON** :

```

CASE gs_result-level.
    WHEN 'S'.
        WRITE / icon_green_light AS ICON.
    WHEN 'W'.
        WRITE / icon_yellow_light AS ICON.
    WHEN OTHERS.
        WRITE / icon_red_light AS ICON.
ENDCASE.
WRITE gs_result-text.

```

- **AS ICON** 告訴清單這個欄位要畫成圖示。
- 圖示在清單上的寬度不一定一樣：表 **ICON** 的 **OLENG** 欄位記錄每個圖示佔幾格 , 例如紅綠燈佔 4 格 , 工具、勾勾佔 2 格。排版欄位座標表時要把這個寬度算進去 (講義 12) 。
- ALV 也能顯示圖示：欄位內容放圖示代碼 , Field Catalog 設 `icon = 'X'` (講義 9) 。

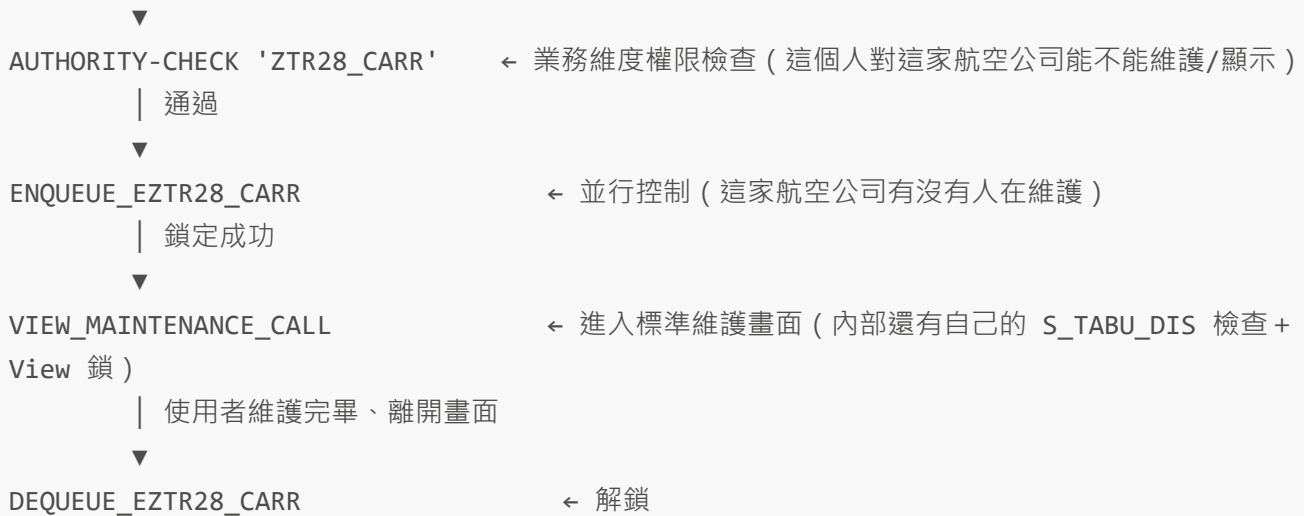
完整示範程式：`ZR_TR28_ICON_DEMO` (`$TMP` , 快照 `zr_tr28_icon_demo.prog.abap`) , 四種用法都在裡面。圖示要在 SAP GUI 上執行才看得到 (SE38 → F8) , 無頭執行只看得到代碼與文字。

8. 整體流程總結

```

使用者在 ZR_TR28_PRICE_CALC 選取畫面按「維護主檔(SM30)」按鈕 ( 或直接執行 T-code
ZTR28_MAINT )
    |
    ▼
CALL TRANSACTION 'ZTR28_MAINT' ← S_TCODE 權限檢查 ( 誰能執行這個 T-code )
    |
    ▼
ZR_TR28_PARAM_MAINT 選取畫面：輸入航空公司、勾選顯示/維護
    |

```



四層防護 (T-code 權限、業務權限、Lock Object、SM30 原生防護) 疊在一起，缺任何一層都可能被繞過——這正是企業客製常見「看似簡單的維護畫面，其實包了好幾層防護」的真實樣貌。

9. 維護出來的參數不是憑空存在：ZR_TR28_PRICE_CALC 計算報表

前面八節都在講「怎麼安全地維護一個折扣值」，但如果這個折扣值維護完之後完全沒有任何程式去讀它、套用它，這一整套 Wrapper / Lock 機制的教學意義會很薄弱——實務上任何一張主檔表存在的理由，幾乎都是「有下游流程要用它算東西」。ZR_TR28_PRICE_CALC 就是示範這個「下游」：

```
PARAMETERS: p_carrid TYPE spfli-carrid DEFAULT 'LH' OBLIGATORY,
             p_connid TYPE spfli-connid DEFAULT '0400' OBLIGATORY.

...
SELECT SINGLE cityfrom cityto
  INTO (lv_cityfrom, lv_cityto)
  FROM spfli
  WHERE carrid = p_carrid AND connid = p_connid.

...
SELECT carrid connid fldate price currency
  INTO CORRESPONDING FIELDS OF TABLE lt_sflight
  FROM sflight
  WHERE carrid = p_carrid AND connid = p_connid
  ORDER BY fldate.

...
SELECT SINGLE discount_pct
  INTO lv_discount_pct
  FROM ztr28_cdisc
  WHERE carrid = p_carrid.
IF sy-subrc <> 0.
  lv_discount_pct = 0.    " 折扣是選配，查不到不擋報表執行
ENDIF.

...
gs_result-final_price = ls_sflight-price * ( 1 - lv_discount_pct / 100 ).
```

三個重點：

1. 這支報表完全不需要走權限/鎖定 **Wrapper**——它只是「讀」折扣值來算價錢，不是「寫」折扣值，所以不需要 **AUTHORITY-CHECK / ENQUEUE**。哪些程式需要包 **Wrapper**、哪些不需要，取決於它是不是真的在「維護」（寫入）受保護的資料，不是看它有沒有用到那張表。
2. 折扣查不到要有合理的預設行為，不能讓報表直接掛掉——很多航空公司可能根本沒有人維護過折扣，這時候用 0%（不打折）繼續往下算，比讓整支報表因為 **SELECT SINGLE** 找不到資料就中斷來得實用。
3. 這是驗證 **Wrapper / DDIC** 設計是否正確的最佳測試工具：在 **ZR_TR28_PARAM_MAINT** 把某家航空公司的折扣改成不同數字，立刻執行 **ZR_TR28_PRICE_CALC** 就能看到 **final_price** 跟著變——如果沒有變，代表維護的資料沒有真的存進 **ZTR28_CDISC**，或是兩支程式讀寫的欄位對不起來，這比單純看 **SM30** 畫面「存檔成功」的訊息更能確認資料真的生效。

9.1 REUSE_ALV_GRID_DISPLAY 在無頭環境的意外行為（以及它的限制）

ZR_TR28_PRICE_CALC 的輸出改用 **REUSE_ALV_GRID_DISPLAY**（Functional ALV，跟講義 9 同一套寫法）。原本預期這種會開 ALV Grid 畫面的呼叫，會跟 **VIEW_MAINTENANCE_CALL** 一樣沒辦法用 ADT 的 **programrun** 無頭驗證；實測發現剛好相反——**programrun** 呼叫時沒有真正的 GUI 對話框環境，**REUSE_ALV_GRID_DISPLAY** 會自動退化成傳統文字清單輸出（欄位標題、資料逐列排開），可以直接在無頭環境驗證欄位/資料是否正確，不需要等使用者到 SAP GUI 才能確認邏輯對不對。這跟 **OOP** 課程教的 **cl_salv_table**（物件導向 ALV）行為不同——**cl_salv_table** 在無頭環境會直接卡住等畫面回應，最終逾時斷線（**RFC_CLOSED**），這也是為什麼基礎課選用 Functional ALV（**REUSE_ALV_GRID_DISPLAY**）而不是 OO ALV：除了跟課程進度一致，還多了「可以無頭驗證」的實用好處。

⚠ 但這個好處不是絕對可靠：後續排錯發現，**ZR_TR28_PRICE_CALC / ZR_TR28_PARAM_MAINT** 都遇過某次呼叫卡住之後、之後不管怎麼改程式碼都持續卡住的情況——確認跟程式碼無關（另建全新測試物件驗證同樣邏輯可正常執行），是 ADT **programrun** 對「這個特定物件」的無頭執行 Session 被卡死，不會自己恢復。詳細診斷過程與方法論見 [.claude/rules/sap-adt-mcp.md](#) 第 38 節。實務上遇到某支程式的 **programrun** 持續 **RFC_CLOSED**，先用一個全新物件重現同樣呼叫方式排除「程式碼問題」，如果新物件正常，就直接請使用者到 SAP GUI 測試，不用繼續在 ADT 這邊除錯。

10. Selection Texts：選取畫面欄位標籤（GUI-only，無 ADT API）

ZR_TR28_PARAM_MAINT / ZR_TR28_PRICE_CALC 選取畫面預設顯示技術欄位名稱（**P_CARRID**、**P_CONNID**、**P_DISP**），要改成中文說明得靠 **Selection Texts**——這跟 Text Symbols 是同一類「程式的顯示文字」設定，一樣沒有 **ADT REST API**，只能在 SE38 手動維護：

1. SE38 開啟程式 → **Goto** → **Text Elements**
2. **Selection Texts** 頁籤
3. 逐一填入欄位對應的說明文字（**ZR_TR28_PARAM_MAINT** 的 **P_CARRID / P_DISP**，**ZR_TR28_PRICE_CALC** 的 **P_CARRID / P_CONNID**，具體文字建議見 [ex28](#) 第六點五部分）
4. 存檔、Activate

這步驟純粹是操作體驗上的改善（選取畫面看得懂中文標籤），不影響任何程式邏輯，也不影響前面幾節教的權限/鎖定/篩選機制。